

ICML 2022 | 把检索数据库变成自动机：让检索式语言模型跑得更快

RetoMaton 在困惑度不变的前提下省去最多 83% 的最近邻搜索 (CMU 与 AWS)

检索增强的语言模型 (retrieval-augmented LM) 之所以准，靠的是一个朴素的技巧：每一步都在一个大型外部数据库 (datastore) 里检索与当前上下文最相似的历史片段，再把这些近邻的“下一个词”融合进预测。kNN-LM 已经证明，这样做能显著降低困惑度 (perplexity)，还能在不重新训练的情况下把模型迁移到新领域。代价是速度。

这个最近邻搜索几乎在每一个 token 上都要触发，而它远比模型自身的一次前向计算慢。实际部署中，真正的开销来自搜索，而不是神经网络本身，这也正是这类效果不错的模型迟迟难以落地的原因。

来自卡内基梅隆大学 (CMU) 与 AWS 的这篇 ICML 2022 论文提出了 RetoMaton (“retrieval automaton”，检索自动机)，直接针对这一开销。作者的观察是：扁平的数据库丢掉了检索本可以复用的结构；他们的做法是把数据库编译成一个加权有限自动机 (weighted finite automaton)，让模型在解码的同时“走”这张自动机。收益是：在困惑度不变的前提下省去最多 83% 的最近邻搜索，或者在保持搜索预算时把困惑度再降最多 1.85。

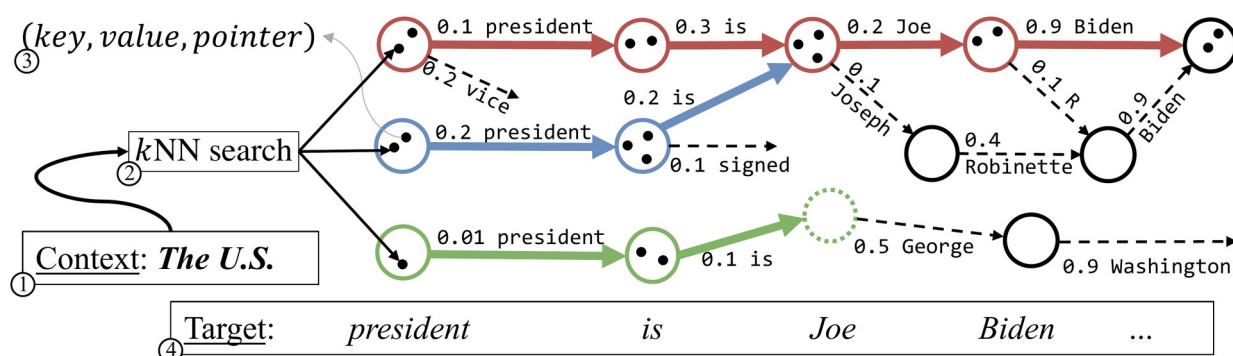


图 1 RetoMaton 的推理过程。从上下文 (“The U.S.”) 出发，一次 kNN 搜索点亮若干自动机状态 (图中彩色圆圈)，每个状态是一簇共享出边指针的数据库条目。随后模型在解码“president is Joe Biden”时并行地沿这些指针前进，把检索结果跨步复用，而不是在每个 token 上从头搜索。虚线箭头表示允许但本次未走的转移。

论文：<https://arxiv.org/abs/2201.12431> · 代码：<https://github.com/neulab/retomaton>

瓶颈：在扁平数据库上反复搜索

kNN-LM 为每个训练 token 存一条记录：一个 key (该位置上 LM 的隐藏状态) 和一个 value (真实出现的下一个词)。测试时它把当前上下文编码成向量，找出最近的 k 个 key，把它们的 value 汇成第二个“下一个词”分布，再与基础 LM 的分布做插值。由于每个 token 一条记录、总量高达数亿条，单次搜索非常昂贵，而模型每生成一个 token 就要搜一次。

作者的关键观察是：这张扁平的列表忽略了两类结构。其一，条目在时间上高度相关：如果某条检索结果此刻有用，那么它在原语料中紧接着的下一条，很可能下一步也有用。其二，key 相近的条目，往往后接同一个词。扁平数据库在每一步都要把这两点从头重新发现。此前的工作如 Adaptive Retrieval (AdaptRet) 学习“何时跳过搜索”，但一旦跳过，它就只能退回到基础 LM、丢掉检索信号，而这恰恰在检索最能帮上忙的领域里最伤性能。

两处改动，把数据库变成自动机

RetoMaton 对数据库做了两处无监督改动，都画在图 2 里。第一，为每条记录保存一个指针，指向它在文本中的后继（紧接其后的那条记录）。第二，把 key 向量相近的条目聚成状态（state），每个状态继承其全部成员的出边指针。指针与聚类一起，把扁平列表变成了一个加权有限自动机：状态是相似上下文的集合，转移则是“下一个是什么”的链接。

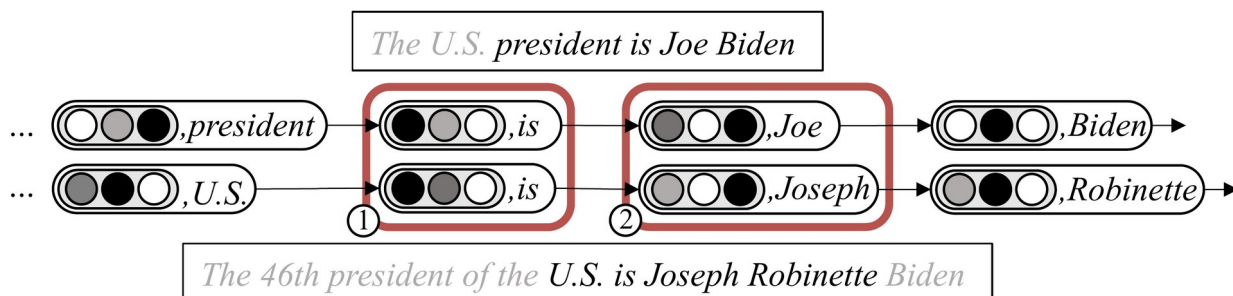


图 2 无监督地构建自动机。两句话共享“... is ...”这段延续。每条记录是一个 (key, value) 对；指针把每条记录连到它在文本中的后继，key 相近的条目被合并进共享状态（1 和 2）。合并让一次搜索覆盖多个上下文，指针则让模型无需再搜索就能向前一步。

推理时，模型维护一小组活跃状态，与 LM 并行地在自动机上前进（图 1）。要向前一步，它只需沿着 value 与刚生成 token 相符的那些条目的指针走，几乎不花代价。转移权重不是固定的：它们由当前隐藏状态与各状态内条目之间的距离动态算出，再与基础 LM 插值，即 $p(w | c, S) = \lambda * p_{\text{auto}}(w | c, S) + (1 - \lambda) * p_{\text{LM}}(w | c)$ ，因此检索仍会随实时上下文自适应。只有当合法后继转移的数量低于阈值 τ 时，才触发一次完整的 kNN 搜索。这个阈值是唯一的旋钮，用来在精度（频繁重启、搜索更多）与速度（很少搜索）之间权衡。构建自动机不需要任何训练数据，无论是用模型自己的训练语料、还是一个全新领域都同样适用。

结果：搜索更少，困惑度持平甚至更好

在 WikiText-103（同域，103M 训练 token，247M 参数的 Transformer 基础 LM，数据库聚成 1M 个状态）上，RetoMaton 在跳过 81% 搜索的同时，困惑度与 kNN-LM 持平。更值得注意的是，即便在 FoSS=0（每一步都搜，和 kNN-LM 一样）时，它仍能把困惑度从 16.65 (kNN-LM) 和 16.35 (AdaptRet) 降到 16.08（表 1）。FoSS (Fraction of Saved Searches，省下搜索的比例) 在这里作为与硬件无关的、对实际墙钟时间节省的近似。

表 1 WikiText-103 同域困惑度，FoSS=0（不省搜索）。越低越好。

模型	困惑度
kNN-LM	16.65
AdaptRet	16.35
RetoMaton (仅指针)	16.12
RetoMaton	16.08

在领域迁移上，收益更大也更稳。用 Law-MT (19M token, 656M 参数基础 LM, 200K 个状态) 构建数据库、把通用模型迁到法律文本后，RetoMaton 在 FoSS=0 时把困惑度从 12.34 (kNN-LM) 降到 10.49。而且随着省下的搜索变多，它的困惑度只是缓慢上升，kNN-LM 却急剧变差，因为 kNN-LM 一旦跳过搜索，除了基础模型就无可依靠。当基础 LM 本身已在 Law-MT 上微调时，RetoMaton 依然带来最大增益 (表 2)：把困惑度从 8.61 降到 7.10，相对降低 17.5%，而 kNN-LM 为 7.93、AdaptRet 为 7.81。

表 2 Law-MT，基础 LM 已在 Law-MT 上微调。两个节省档位下的困惑度，越低越好。

模型	FoSS=0	FoSS=0.5
微调后 LM	8.61	-
kNN-LM	7.93 (-7.9%)	8.25 (-4.2%)
AdaptRet	7.81 (-9.2%)	7.91 (-8.1%)
RetoMaton	7.10 (-17.5%)	7.15 (-17.0%)

括号内为相对微调后 LM 的降幅。即便省去一半搜索，RetoMaton 仍保住几乎全部增益。

到底是谁在起作用：指针还是聚类？

一项消融把两种成分拆开看。只用指针、完全不做聚类，就已经击败所有基线，并在省下超过 60% 搜索的同时与 kNN-LM 持平，可见在低节省率下，“下一个是什么”的链接贡献了大部分收益：仅指针版本在 FoSS=0 时达到 16.12 的困惑度，与完整模型的 16.08 几乎打平。聚类主要在高 FoSS (大约 0.7 及以上) 时发挥作用，此时合并后的状态让模型能在更长的文本片段上保持无需搜索。至于聚类数量，500K 与 1M 个状态表现相近，100K 则过于粗糙；更省成本的贪心合并并在 FoSS=0 时占优，但随节省比例增大而逐渐落后。

为什么重要

这项工作留下的核心思路是：检索数据库本身有值得“编译”的结构。通过用指针把相邻条目串起来、把相似条目聚成自动机状态，RetoMaton 让语言模型把检索沿时间向前携带，而不必在每个 token 上重付一次搜索。它的构建是无监督的、与模型无关、可跨领域迁移，并在一个动态机制里统一了 token 级、chunk 级与序列级检索。

也要看清边界：RetoMaton 降低的是搜索的频率，而不是单次搜索的代价，也不是存下数亿条记录所需的内存，数据库本身仍要有地方安放。节省是通过 FoSS 来衡量的，它是对墙钟时间的近似，而非某个固定系统上的实测时间；而且自动机需要先对每个数据库构建一次，之后才开始“回本”。在这些限制之内，它是一种干净、无需训练、能让检索式语言模型运行成本大幅下降的方法。